

ENGSE206 · การกำหนดความต้องการและการออกแบบทางซอฟต์แวร์

วิศวกรรมซอฟต์แวร์ · มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี (ดอยสะเก็ด)

สัปดาห์ที่ 6

Model Behavior and Make It Testable

User Story · Use Case Lite · Acceptance Criteria



ทีมนักศึกษา/BA อธิบายพฤติกรรมระบบร่วมกับผู้เกี่ยวข้อง ก่อนแปลงเป็น model ที่ตรวจได้

เอกสารนี้คืออะไร

เอกสารประกอบการสอนฉบับอ่านเองได้ สำหรับทฤษฎี 2 ชั่วโมง + Lab/Workshop 3 ชั่วโมง ยึดสัญญาณงาน W06-LITE-v1.0

คำถามหลักของสัปดาห์: จะสื่อสารว่า “ใครต้องการทำอะไร”, “ระบบตอบสนองอย่างไร” และ “ตรวจว่าทำถูกต้องอย่างไร”?

0 ภาพรวมสัปดาห์และตำแหน่งในรายวิชา

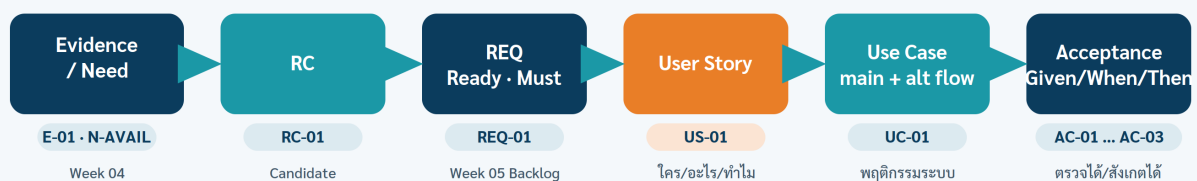
Week 06 เป็นสัปดาห์ที่สองของระยะ Define/Specify (Week 05–08) เป้าหมายคือนำ requirement ชุดเล็กที่ “พร้อมใช้ (Ready)” จาก Backlog v0.1 มาอธิบายเป็นแบบจำลองพฤติกรรม (behavior model) ที่สื่อสารได้ ตรวจสอบย้อนกลับได้ และทดสอบได้ โดยยังไม่เขียน SRS ทั้งฉบับและยังไม่ออกแบบระบบ

Week 05 ส่งอะไรมาให้ และ Week 06 ส่งอะไรต่อ

จาก Week 05 (Input)	Week 06 ทำ	ส่งต่อ Week 07 (Output)
[object Object]	[object Object]	[object Object]

สายการสืบย้อน (Trace Chain) — อย่างน้อย 1 เส้นต่อทีม

ทุกสิ่งก็ต้องรักษาความหมาย · ห้ามเพิ่ม actor / business rule ที่ไม่มีในต้นทาง



กติกาการ trace

1) ทุก US / UC / AC ต้องชี้กลับ REQ ได้ 2) REQ ต้องชี้กลับ Evidence/Need ได้ 3) หากพบข้อเท็จจริงใหม่ระหว่างทำ model ให้บันทึกเป็น Open Question ไม่ใช้การ “สร้าง” ข้อกำหนดใหม่ · Priority สูงไม่ได้แปลว่าพร้อม หาก Status ยังเป็น Needs Follow-up / Hold

สายการสืบย้อน (trace chain): Evidence/Need → RC → REQ → User Story → Use Case → Acceptance Criteria

Core vs Extension — ทำแค่ไหนจึงได้คะแนนเต็ม

Core (ทุกทีมทำ / ได้เต็มได้)	Extension (ทำเมื่อ Core พร้อม)
2 User Stories (role/goal/value + source REQ)	story splitting patterns
1 Use Case Lite (actor, goal, precondition, trigger)	fully dressed use case (ทุกฟิลด์)
Main Flow 5–7 ขั้น สลับ actor/system	include / extend, หลาย diagram
Alternate Flow ≥ 1 (step + condition + response)	BDD automation / complete test suite
Acceptance Criteria 3 ข้อ (Given/When/Then)	domain/activity model เกินจำเป็น
Trace ≥ 1 chain	negotiation board

เส้นแบ่งสำคัญ

Core วัดที่ “สื่อสารพฤติกรรมถูกระดับ + ตรวจสอบได้ + trace ได้” ไม่ใช่ “ใช้โน้ตเขียนครบทุกแบบ” — การเพิ่ม include/extend หรือ diagram หลายชั้นไม่ได้ทำให้ได้คะแนนมากขึ้นในสัปดาห์นี้

1 User Story — บอกว่าใครต้องการอะไร เพื่ออะไร

User Story คือประโยคสั้นที่จับ “คุณค่าของผู้ใช้” ไม่ใช่รายละเอียดขั้นตอนหรือหน้าจอ

ใช้เพื่อเปิดบทสนทนาและจัดลำดับงาน รูปแบบมาตรฐานคือ:

ในฐานะ [role] ฉันต้องการ [goal] เพื่อ [value]

องค์ประกอบ 3 ส่วน

ส่วน	ตอบคำถาม	ตัวอย่าง (Campus Resource Booking)
role (ใคร)	ใครได้ประโยชน์ / เป็น actor จริงในต้นทาง	Student Requester
goal (อะไร)	ต้องการทำอะไรในระดับคุณค่า	ค้นหาทรัพยากรที่ว่างตามวัน/เวลา
value (เพื่ออะไร)	ทำไมถึงมีคุณค่า	เพื่อเลือกตัวเลือกก่อนส่งคำขอ

เกณฑ์คุณภาพอย่างง่าย (จำ 3 ข้อพอ)

- มี actor จริง — role ต้องมาจาก backlog/evidence ไม่ประดิษฐ์ stakeholder ใหม่
- เป็นคุณค่า ไม่ใช่ UI — “ค้นหาทรัพยากรว่าง” ไม่ใช่ “กดปุ่มค้นหาสีฟ้า”
- trace ได้ — แต่ละ story ซ้ำกลับ REQ ที่ Ready ได้อย่างน้อย 1 ข้อ



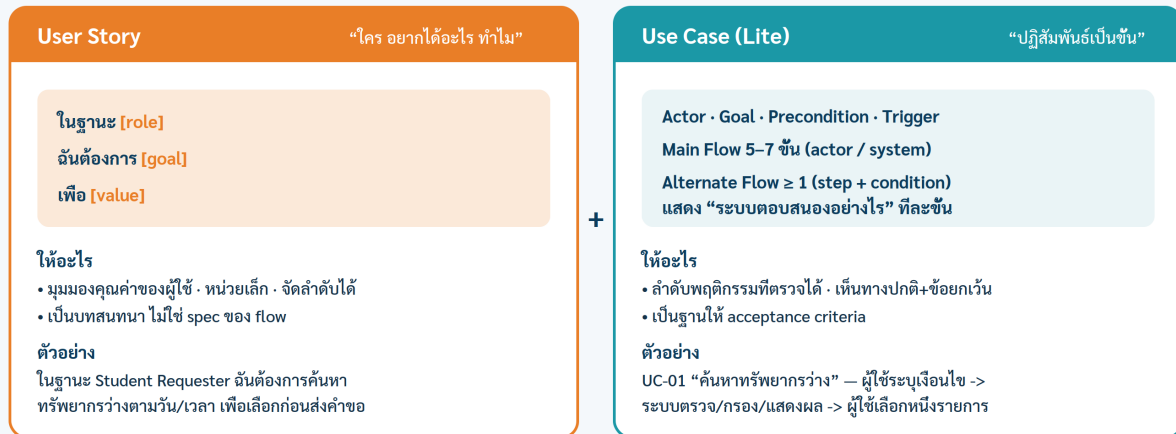
คู่ทำงาน BA/นักศึกษา ร่างการ์ด story จาก need ของผู้ใช้ ก่อนลงรายละเอียด flow

ระวังความเข้าใจผิดที่พบบ่อย

User Story ไม่ใช่ “Use Case แบบย่อ” — story ตอบว่า ใคร/อะไร/ทำไม ส่วน use case อธิบาย ปฏิสัมพันธ์เป็นขั้น ทั้งสองใช้คู่กัน ไม่แทนกัน

User Story ≠ Use Case แบบย่อ

สองสิ่งนี้ตอบคนละคำถาม และใช้คู่กัน ไม่ใช่แทนกัน



หนึ่ง story อาจถูกอธิบายด้วยหนึ่ง use case flow และตรวจด้วยหลาย acceptance criteria

เปรียบเทียบ User Story กับ Use Case Lite — ตอบคนละคำถาม และใช้ร่วมกัน

2 Use Case Lite — อธิบายพฤติกรรมทีละขั้น

Use Case Lite อธิบายลำดับปฏิสัมพันธ์ระหว่าง actor กับระบบเพื่อบรรลุ goal หนึ่ง โดยเก็บเฉพาะข้อมูลที่จำเป็น (ยังไม่ใช่ว่า fully dressed use case)

หัวข้อของ Use Case Lite

ฟิลด์	ความหมาย	ตัวอย่าง (UC-01)
Use Case	ชื่อแบบ verb + object	ค้นหาทรัพยากรว่าง
Primary Actor	ผู้เริ่มและได้ประโยชน์	Student Requester
Goal	ผลลัพธ์ที่ต้องการ	เห็นรายการทรัพยากรที่ว่างตามเงื่อนไข
Preconditions (≤ 2)	เงื่อนไขก่อนเริ่ม	เข้าถึงระบบได้; availability ถูกโหลด
Trigger	เหตุการณ์เริ่มต้น	ต้องการหาทรัพยากรของวัน/ช่วงเวลาหนึ่ง

Main Flow ต้อง “ถูกระดับ” (5–7 ขั้น)

เขียนสลับกันระหว่าง actor action กับ system response — อธิบายว่า “ระบบทำอะไรให้ actor” ไม่ใช่ “หน้าจทำงานอย่างไร”

Main Flow ต้องอยู่ “ระดับที่ถูก”

สลับ Actor action / System response · 5–7 ขั้น · ไม่ใช้รายการปุ่ม/หน้าจอ

✓ ถูกระดับ — พฤติกรรมที่สังเกตได้

- 1 ผู้ใช้ระบุวัน เวลา และประเภททรัพยากรที่ต้องการ
- 2 ระบบตรวจสอบว่าเงื่อนไขค้นหาครบและช่วงเวลาถูกต้อง
- 3 ระบบตรวจสอบสถานะว่างของทรัพยากรที่ตรงเงื่อนไข
- 4 ระบบตัดรายการที่ถูกยืนยันแล้วในช่วงเวลาซ้อนออก
- 5 ระบบแสดงรายการว่างพร้อมข้อมูลที่จำเป็นต่อการเลือก
- 6 ผู้ใช้เลือกทรัพยากรหนึ่งรายการเพื่อดำเนินการต่อ

Actor action

System response

✗ ผิดระดับ — เป็น UI steps

- ผู้ใช้คลิกปุ่ม “ค้นหา” สีฟ้า
- เปิด dropdown เลือกห้อง
- กรอกช่องวันที่ในฟอร์ม
- ระบบโหลด spinner แล้วเด้ง modal

เหตุผล: บอก “หน้าจทำงานอย่างไร” แทนที่จะบอก “ระบบทำอะไรให้ actor”
-> ผูกกับดีไซน์ ยังไม่ถึงเวลานี้

เคล็ดตรวจระดับ

ถ้าเปลี่ยนดีไซน์หน้าจอแล้วขั้นตอนยังจริงอยู่ = ถูกระดับ · ถ้าขั้นตอนพังเพราะเปลี่ยนปุ่ม/เมนู = เขียนดำเนินไป (UI step)
Alternate flow ต้องผูกกับ “step + condition” และบอก system response เสมอ (เช่น At step 5 · ไม่พบรายการว่าง)

ตัวอย่าง main flow ที่ถูกระดับ เทียบกับตัวอย่างที่ผิดเพราะเป็น UI steps

เคล็ดตรวจระดับ

- ถ้าเปลี่ยนดีไซน์หน้าจอแล้วขั้นตอนยัง “จริง” อยู่ = ถูกระดับ
- ถ้าขั้นตอนพังเพราะเปลี่ยนปุ่ม/เมนู = เขียนตำเกินไป (UI step)

Alternate / Exception Flow (อย่างน้อย 1)

ต้องผูกกับ step + condition ที่ชัด และบอก system response เช่น

At step: 5

Condition: ไม่พบทรัพยากรว่างตามวัน/เวลาที่ระบุ

System response: ระบบแจ้งว่าไม่พบรายการว่าง และให้ผู้ใช้แก้ไขเงื่อนไข โดยไม่แสดงทรัพยากรที่ถูกยืนยันแล้วว่า “ว่าง”



เวิร์กช็อปเรียงการ์ด flow เป็นลำดับ actor  system เพื่อคุมระดับก่อนพิมพ์ลงเอกสาร

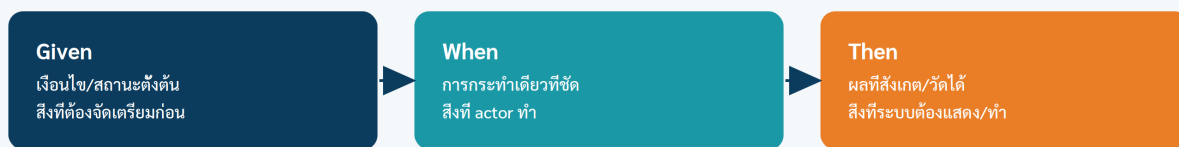
3 Acceptance Criteria — ทำให้ผลลัพธ์ตรวจได้

Acceptance Criterion (AC) บอกว่า “ทำถูก” หน้าตาเป็นอย่างไร ในรูปแบบที่ผู้ทดสอบตัดสินใจผ่าน/ไม่ผ่าน ได้ ใช้โครง

Given / When / Then

Acceptance Criteria = เงื่อนไข + การกระทำ + ผลที่สังเกตได้

รูปแบบ Given / When / Then ทำให้ผู้ทดสอบตัดสินใจผ่าน/ไม่ผ่าน ได้



ตัวอย่างที่ดี (AC-01 · trace REQ-01, REQ-04)

Given ห้อง A ถูกยืนยันแล้วเวลา 13:00–15:00

When ผู้ใช้ค้นหาช่วงเวลาเดียวกัน

Then ห้อง A ไม่ปรากฏในรายการ “ว่าง”

✗ **กำกวม — วัดไม่ได้**

“ระบบใช้งานง่าย”

“ค้นหาเร็ว” · “แสดงข้อมูลถูกต้อง”

ไม่มีเงื่อนไข ไม่มีผลที่สังเกตได้ -> เกือบกันไม่จบ

✓ **สังเกตได้ — ตรวจได้**

“ผลค้นหาเฉพาะรายการที่ยังไม่ถูกยืนยันในช่วงเวลาที่เลือก”

มีเงื่อนไข + การกระทำ + ผลที่ผู้ทดสอบเห็นได้

กายวิภาคของ AC: Given (เงื่อนไข) → When (การกระทำ) → Then (ผลที่สังเกตได้) และตัวอย่างกำกวมเทียบสังเกตได้

ส่วน	คือ	ต้องมี
Given	เงื่อนไข/สถานะตั้งต้น	สิ่งที่ต้องจัดเตรียมก่อน
When	การกระทำเดียวที่ชัด	สิ่งที่ actor ทำ
Then	ผลที่สังเกต/วัดได้	สิ่งที่ระบบต้องแสดง/ทำ

ตัวอย่างที่ดี (AC-01 · trace REQ-01, REQ-04)

Given ห้อง A ถูกยืนยันแล้วเวลา 13:00–15:00

When ผู้ใช้ค้นหาช่วงเวลาเดียวกัน

Then ห้อง A ไม่ปรากฏในรายการ “ว่าง”

อย่าเขียนแบบนี้ (กำกวม วัดไม่ได้)

- “ระบบใช้งานง่าย” · “ค้นหาเร็ว” · “แสดงข้อมูลถูกต้อง”

ไม่มีเงื่อนไข ไม่มีผลที่สังเกตได้ → เกือบกันไม่จบ และผู้ทดสอบตัดสินใจไม่ได้



ทีมตรวจสอบผ่าน/ไม่ผ่านบนหน้าจอ (แถบเขียว/แดง) เทียบกับ Given/When/Then บนกระดาษ

4 Traceability และ Use Case Overview Diagram

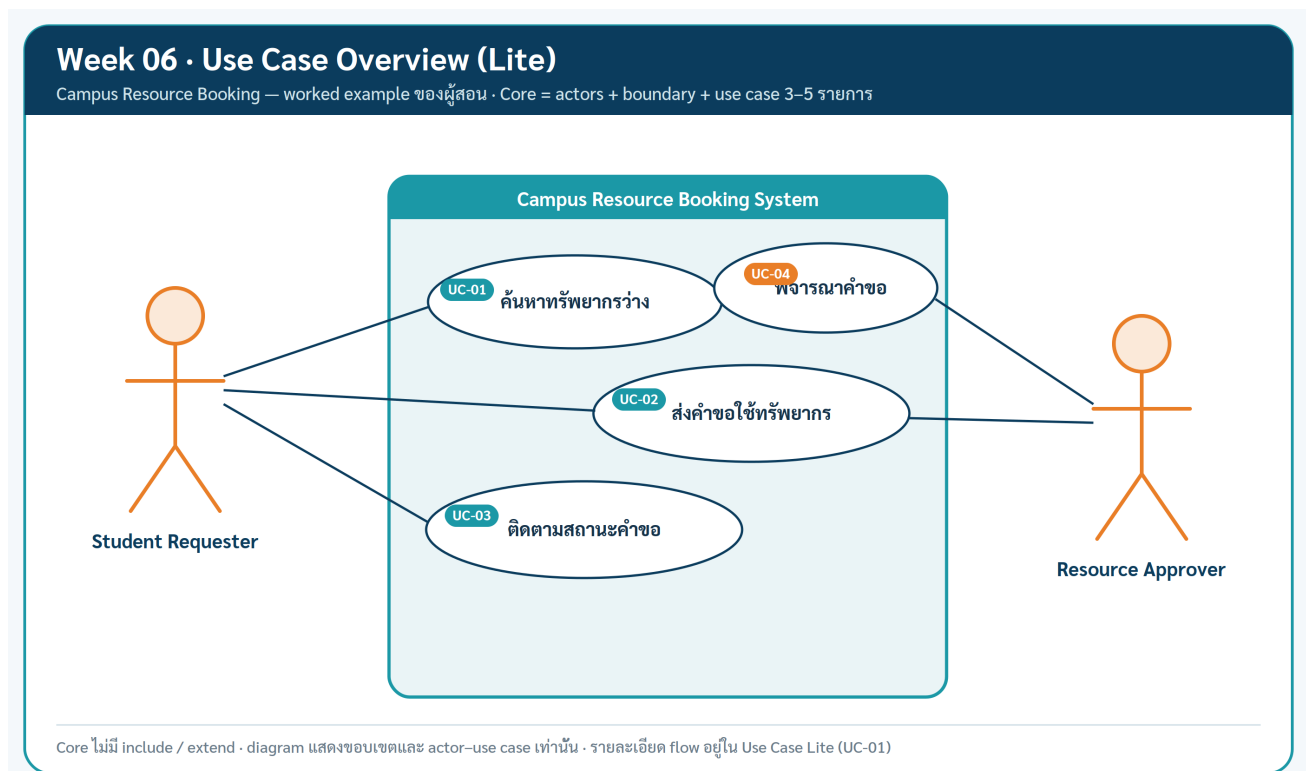
ทุก model ต้อง trace กลับไปยัง requirement และหลักฐานได้อย่างน้อย 1 เส้น เพื่อรับประกันว่าไม่ได้ “สร้าง” ข้อกำหนดใหม่ระหว่างทำ model

ตาราง Trace (ตัวอย่าง Campus Resource Booking)

Story / Use Case / AC	Requirement	Evidence
US-01 / ค้นหาทรัพยากรว่าง (UC-01) / AC-01-AC-03	REQ-01, REQ-04	E-01, E-03, F-03
US-02 / ส่งคำขอใช้ทรัพยากร	REQ-02	E-04

Use Case Overview (Core)

diagram แสดงเฉพาะ actors + system boundary + use case หลัก 3-5 รายการ ไม่มี include/extend และต้องตรงกับ actor/use case ที่เขียนไว้



Use Case Overview (Lite) ของ Campus Resource Booking — 2 actors, 1 boundary, 4 use cases

กฎของ diagram

- diagram ไม่แทน use case specification — รายละเอียด flow อยู่ในข้อความ
- ห้ามเพิ่ม actor/behavior ที่ไม่มีใน backlog/evidence
- ถ้าต้องแม่นยำ ให้วาดเป็น vector/shape ไม่ใช่ภาพจาก AI

5 ตัวอย่างเดินครบวงจร — Campus Resource Booking

ต่อไปนี้เป็น worked example ของผู้สอน (นักศึกษาทำ Case ID ของตนเอง 01–10 ด้วยโครงสร้างเดียวกัน) แสดงการแปลง REQ-01/REQ-02/REQ-04 เป็น model pack

เลือก requirement และ Stories

Story	ข้อความ	Source REQ
US-01	ในฐานะ Student Requester ฉันต้องการค้นหาทรัพยากรที่ว่างตามวันและเวลา เพื่อเลือกตัวเลือกก่อนส่งคำขอ	REQ-01
US-02	ในฐานะ Student Requester ฉันต้องการส่งข้อมูลวัน เวลา ทรัพยากร และวัตถุประสงค์ เพื่อให้คำขอเข้าสู่การพิจารณา	REQ-02

Use Case UC-01 “ค้นหาทรัพยากรว่าง” — Main Flow

ขั้น	Actor action / System response
1	ผู้ใช้ระบุวัน เวลา และประเภททรัพยากร
2	ระบบตรวจสอบว่าข้อมูลค้นหาครบและช่วงเวลาถูกต้อง
3	ระบบตรวจสอบ availability ของทรัพยากรที่ตรงเงื่อนไข
4	ระบบตัดรายการที่มีคำขอยืนยันแล้วในช่วงเวลาซ้อนออก
5	ระบบแสดงรายการที่ว่างพร้อมข้อมูลจำเป็นต่อการเลือก
6	ผู้ใช้เลือกทรัพยากรหนึ่งรายการเพื่อดำเนินการต่อ

Acceptance Criteria

AC	Given / When / Then	REQ
AC-01	Given ห้อง A ถูกยืนยันแล้ว 13:00–15:00 · When ค้นหาช่วงเดียวกัน · Then ห้อง A ไม่ปรากฏใน “ว่าง”	REQ-01,04
AC-02	Given ห้อง B ว่าง 13:00–15:00 ตรงประเภท · When ค้นหาเงื่อนไขนั้น · Then ห้อง B ปรากฏพร้อมวัน/เวลา	REQ-01
AC-03	Given ยังไม่ระบุเวลา · When กดค้นหา · Then ระบบไม่ค้นหาและชี้ช่องที่ต้องกรอก	REQ-01

Revision Note (เห็นการแท้จริง)

เพิ่ม AC-01 จาก “ระบบแสดงข้อมูลถูกต้อง” เป็น scenario ห้อง A และช่วงเวลาเฉพาะ
เพื่อให้ผู้ทดสอบเห็นเงื่อนไขและผลผ่าน/ไม่ผ่านได้

6 สรุปความเข้าใจผิดที่ต้องระวัง

ความเข้าใจผิด	วิธีคิดที่ถูกต้อง
User Story = Use Case แบบย่อ	คนละคำถาม; story = ใคร/อะไร/ทำไม, use case = ปฏิสัมพันธ์เป็นขั้น
AC คือการทวน requirement	AC ต้องมี condition/action/observable result ที่ตรวจได้
“ใช้งานง่าย/เร็ว/ถูกต้อง” เป็น AC ได้	ต้องเปลี่ยนเป็น scenario ที่สังเกตผลได้
Main flow = รายการปุ่ม/หน้าจอ	ต้องสลับ actor action/system response ระดับพฤติกรรม
Alternate flow เขียนลอย ๆ ได้	ต้องผูก step + condition และบอก system response
Diagram แทน specification ได้	diagram เสริม text; ห้ามเพิ่ม actor/behavior นอกต้นทาง
Priority สูง = พร้อมใช้	ถ้า status ยัง Needs Follow-up/Hold ยังไม่พร้อมทำ model

7 Responsible AI และ Two-Repository Workflow

AI เป็น simulation/assistant ไม่ใช่แหล่งข้อเท็จจริง ใช้ช่วยตรวจ/ตั้งคำถาม/เสนอถ้อยคำ/ชี้ความกำกวมได้ แต่ห้ามประดิษฐ์ fact, policy, actor หรือ business rule และต้องบันทึก AI use log

บันทึกใน AI use log	ตัวอย่าง
1) สั่งให้ AI ทำอะไร	ช่วยตรวจว่าประโยค AC ข้อใดยังกำกวม
2) ใช้/ปฏิเสธ output ไດ	รับคำแนะนำ AC-01, ปฏิเสธ AC ที่ AI แต่งเงื่อนไขเกินหลักฐาน
3) ตรวจกับ source/evidence ไດ	เทียบกับ E-03/F-03 ใน evidence log
4) มนุษย์ทำอะไร & ทำไม	เขียน scenario ห้อง A เองเพื่อให้ตรวจได้

รูปแบบการทำงาน: READ จาก Course Repo → WORK และ SUBMIT ใน Team Repo (ห้ามแก้ Course Repo, ห้ามใส่เฉลย/คะแนนลง Student Template)

READ จาก Course Repo -> WORK และ SUBMIT ใน Team Repo

ห้ามแก้ Course Repo · ห้ามใส่เลข/คะแนน/ไฟล์ผู้สอนลง Student Template

Course Repo — READ ONLY

- คำสั่งสัปดาห์ · rubric · assignment contract
 - template ของงาน
templates/06-requirement-models.md
 - completed example (Campus Resource Booking)
examples/campus-resource-booking/week-06/
 - Case Card 01–10 (ทีมใช้ Case ID ของตน)
 - prompt pack สำหรับ Responsible AI
- เจ้าของ/แก้ไข: ผู้สอนเท่านั้น

คัดลอก
ไม่แก้ต้นฉบับ

Team Repo — WORK & SUBMIT

- งานจริงของทีม (Case ID ของตนเอง)
docs/06-requirement-models.md
diagrams/use-case/w06-use-case-overview.svg
submissions/week-06-submission.md
 - evidence · worklog · decision log
 - AI use log + human verification
 - commit: **submit(w06): beginner requirement models**
- เจ้าของ/แก้ไข: ทีมนักศึกษา

READ จาก Course Repo → WORK/SUBMIT ใน Team Repo พร้อม path และ commit ของสัปดาห์



ทีมงานทำงานร่วมกันใน Team Repo ของตนเอง โดยใช้ Case ID ของกลุ่ม

8 การประเมิน · Definition of Done · Exit Ticket

Rubric 10 คะแนน

เกณฑ์	คะแนน	หลักฐาน
Actor-goal-value alignment	3	2 stories เชื่อม stakeholder need/backlog
Use case flow	3	main 5–7, alternate ≥ 1 , boundary ชัด
Acceptance criteria and trace	2	AC 3 สังเกตผลได้และ trace REQ/evidence
Review, revision and submission	2	peer review, revision, diagram/commit/exit ticket

Definition of Done

- มี 3 artefact ครอบคลุมทุก path และลิงก์ใช้งานได้
- Core count ครอบคลุม (2 stories · 1 use case · main 5–7 · alternate ≥ 1 · AC 3 · trace ≥ 1) โดยไม่บังคับ Extension
- story/flow/AC trace ถึง Ready REQ และ evidence; ทุก AC มี setup/action/observable result
- diagram สอดคล้องกับ actor/use case ที่เขียน; มี peer review + revision ที่เห็นได้ 1 จุด
- มี AI use log + human verification; individual exit ticket; บันทึก final commit hash และ push แล้ว

Individual Exit Ticket

เลือก AC 1 ข้อ แล้วอธิบายว่า ผู้ทดสอบต้อง (1) เตรียมอะไร [Given] (2) กระทำอะไร [When] (3) สังเกตอะไรจึงตัดสิน “ผ่าน/ไม่ผ่าน” [Then]

9 แผนการสอน — ทฤษฎี 120 นาที + Lab 180 นาที

ทฤษฎี 120 นาที

เวลา (นาที)	โหมด	กิจกรรม	Check
20	Connect	W05 Must/Ready → actor/goal/value	story skeleton
35	I Do	Think-aloud: REQ → Story → Use Case → AC	1 full trace chain
35	We Do	เรียง actor/system flow cards; ตัด UI detail	main flow ถูกระดับ
30	We Do	เปลี่ยน “ง่าย/เร็ว/ถูกต้อง” เป็น observable AC	pass/fail examples

Lab / Workshop 180 นาที

เวลา (นาที)	งานของทีม	Checkpoint
35	เขียน 2 stories จาก top backlog	actor/goal/value/source
45	Use case: main 5–7 + alternate ≥1	actor action/system response
35	เขียน AC 3 แบบ Given/When/Then	observable result
35	Peer trace review	Actor→Goal→Flow→AC→REQ→ Evidence
30	Revise, export diagram, submit	baseline v0.1 + commit

ก่อนจบคาบ

เลือก AC 1 ข้อของทีม แล้วถามผู้ทดสอบสมมุติว่าจะรู้ “ผ่าน/ไม่ผ่าน” อย่างไร ถ้ายังตอบไม่ได้ ให้แก้ expected result ก่อนทำ diagram และ commit `submit(w06): beginner requirement models`